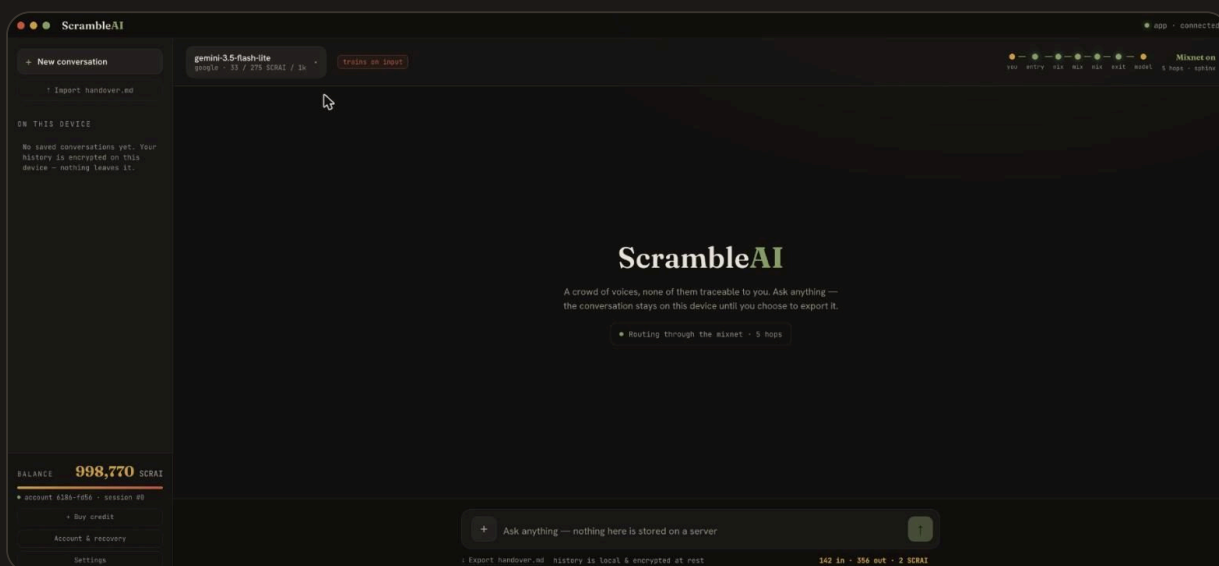


PITCH · STAND 27. AUGUST 2026

ScrambleAI

Anonyme KI für alle. Jede Frage an einen Chatbot verrät zwei Dinge: **wer** fragt und **was** gefragt wird. ScrambleAI trennt beides — technisch, nicht per Versprechen: Fragen laufen durchs **Nym-Mixnet**, bezahlt wird mit blind signierten **Coconut Credentials** (zk-nym), und die Historie verlässt das Gerät nie.

**5 Hops**

SPHINX-ROUTING

~1,1 s

MIXNET-ANLAUF, REAL GEMESSEN

-60 %

TOKENKOSTEN DURCH PRUNING

4K

BILD ÜBER SURBS GELIEFERT, GEMESSEN

zk-nym

COCONUT CREDENTIALS

100 % Rust

CLIENT-KERN + SERVER

Alle Screenshots in diesem Dokument sind echte Aufnahmen des laufenden MVP — keine Mockups. Jedes Bild lässt sich per Klick in voller Größe öffnen.

© 2026 Hermes Blockchain Ventures — sämtliche Konzepte, Code und Gestaltung in diesem Dokument sind geistiges Eigentum der Hermes Blockchain Ventures.

Genutzte Open-Source-Lizenzen: Nym SDK & nym-compact-ecash (Apache-2.0) · Tauri (MIT/Apache-2.0) · Rust-Crate-Ökosystem u. a. tokio, serde, reqwest (MIT/Apache-2.0) · ed25519-dalek (BSD-3-Clause) · SQLite (Public Domain) · BTCPay Server (MIT) · GLiNER & Tesseract (Apache-2.0) · Schriften Fraunces, Hanken Grotesk, JetBrains Mono (SIL OFL 1.1).

Jede KI-Frage ist ein Geständnis

Gesundheitsfragen, Vertragsentwürfe, Bewerbungen, Beziehungsprobleme — was Menschen einer KI anvertrauen, ist intimer als jede Suchhistorie. Heute landet all das mit Klarnamen-Account, IP-Adresse und Zahlungsdaten bei einem Anbieter, der auf den Inhalten trainieren darf.

Bestehende „anonyme“ Zugänge lösen nur die Hälfte: ein Proxy sieht selbst alles, ein VPN-Anbieter kennt seine Kunden, und bezahlt wird am Ende doch mit Kreditkarte. ScrambleAI ist so gebaut, dass **keine einzelne Partei „wer“ und „was“ gleichzeitig sieht** — auch nicht wir selbst. Der Betreiber *kann* nicht neugierig sein.

Warum das für Nym zählt

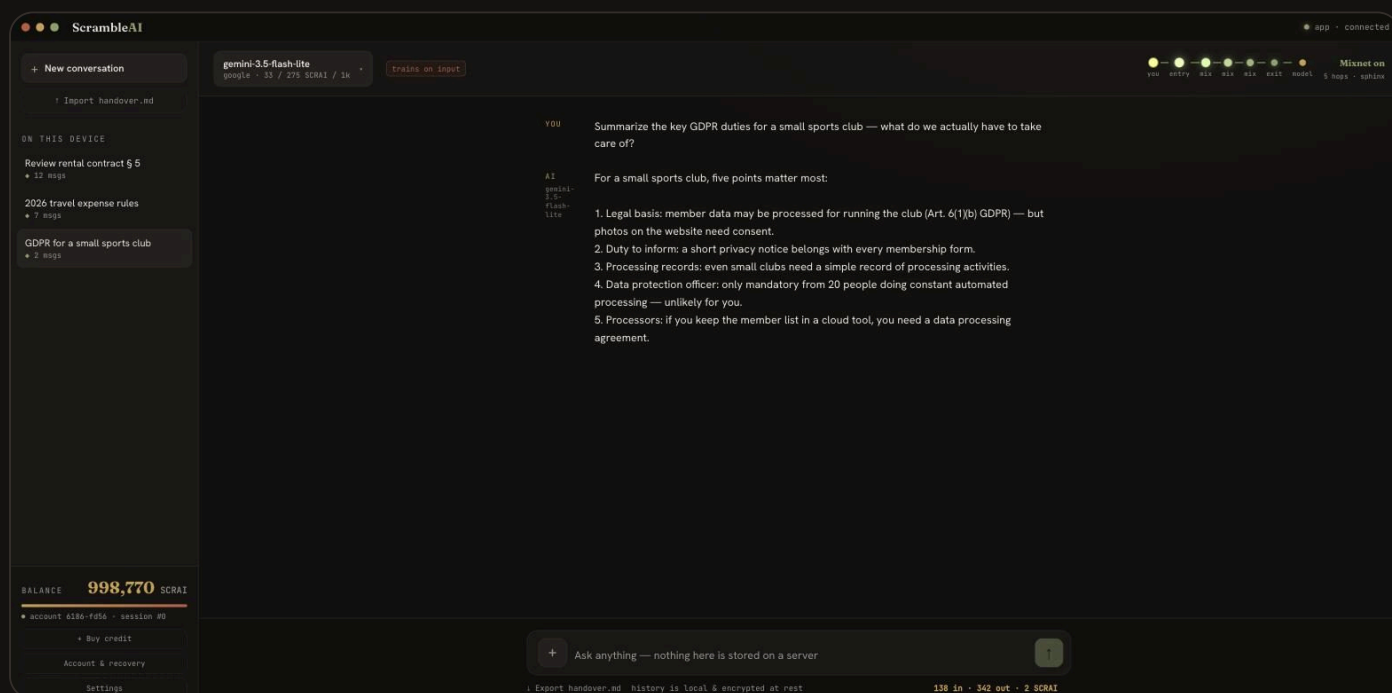
Mixnet-Adoption hat ein Henne-Ei-Problem: kein Traffic ohne Anwendungen, keine Anwendungen ohne Traffic. ScrambleAI könnte **die erste kommerzielle Endkunden-App auf dem Nym-Mixnet** werden — ein System-Seller, leicht unbescheiden formuliert: eine App, die Menschen wegen des Produkts installieren (private KI, die geständnisreichste Workload überhaupt) — und die als Nebeneffekt echten Sphinx-Traffic, echte SURB-Nutzung und mit jedem Aufladen native NYM-Nachfrage auf der Nyx-Chain erzeugt.

Ein Chat wie jeder andere — über einen Pfad wie kein anderer

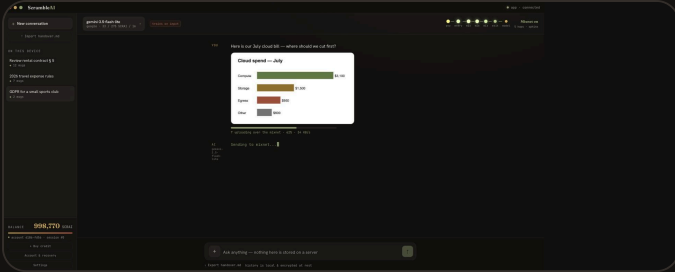
ScrambleAI ist eine Desktop-App (mobil folgt) mit eingebettetem Nym-Client. Jede Nachricht wird in Sphinx-Pakete verpackt und über fünf Stationen geroutet: Entry-Gateway, drei fremde Mix-Knoten, Exit. Der Server antwortet über anonyme Reply-Blöcke (SURBs) — er kennt keine Absenderadresse, an die er sich erinnern könnte.

Anhänge — Bilder und PDFs — nehmen denselben Weg: Große Dateien sind als eine Mixnet-Nachricht unzuverlässig, deshalb zerlegt die App sie in **quittierte Chunks** mit echtem Fortschritt pro Chunk und Live-Speed, direkt auf dem Bild angezeigt — jederzeit abbrechbar — der Server setzt sie erst für den Request zusammen, der sie referenziert. Die Konversation selbst liegt ausschließlich lokal, verschlüsselt mit AES-256-GCM; Export und Umzug laufen über ein offenes, portables Format (`handover.md`), auf Wunsch passphrase-verschlüsselt.

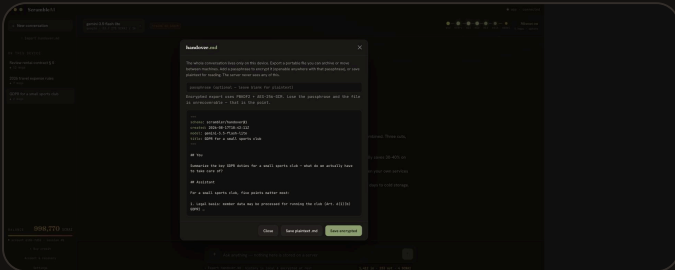
Antworten skalieren genauso. Ein generiertes Bild ist keine einzige riesige Mixnet-Nachricht: Der Server parkt es und antwortet mit einer kleinen Referenz, die App holt es in **96-KB-Chunks**, **acht gleichzeitig** zurück und zeigt einen echten Fortschrittsbalken. Ein hängender Chunk wird allein neu angefordert; ein abgebrochener Download wird **per Retry fortgesetzt** — nur die fehlenden Chunks, keine zweite Generierung, keine zweite Abbuchung. Auf dem iPhone gemessen: ein **4K-Bild** (5632×3072), 46 Chunks, Ende zu Ende durchs Mixnet in rund vier Minuten, abgerechnet zu Googles Listenpreis plus Marge.



Der Chat. Oben rechts die echte Route: you · entry · mix · mix · mix · exit · model. Links die verschlüsselte lokale Historie. Unten rechts der Preis der Antwort — 138 in · 342 out · 2 SCRAI.



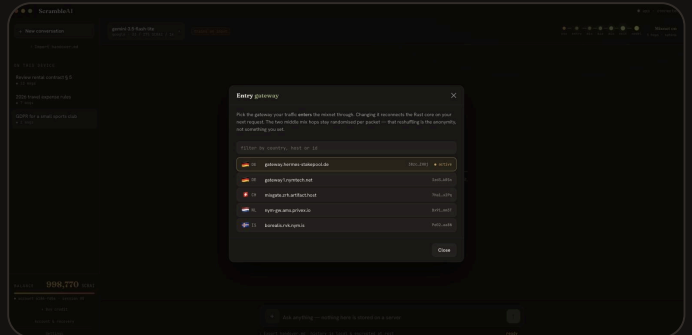
Ein Vision-Request unterwegs. Das angehängte Bild streamt in quittierten Chunks hoch („uploading over the mixnet · 62% · 34 KB/s“), während die Statuszeile den echten Lebenszyklus tippt – „Sending to mixnet...“ mit Live-Cursor.



Deine Daten, portabel. Ein Klick exportiert die Konversation als offenes Markdown – im Klartext zum Lesen, oder PBKDF2 + AES-256-GCM-verschlüsselt für den Umzug.



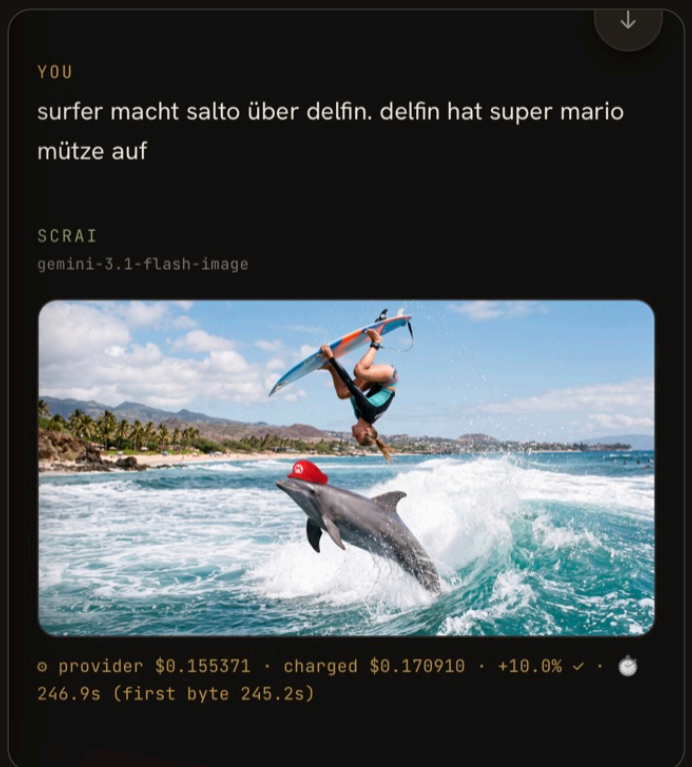
Die Antwort, Ende zu Ende. Das Modell liest das angehängte Chart über denselben anonymen Pfad; der Footer bepreist den multimodalen Request – 1,412 in · 233 out · 4 SCRAI.



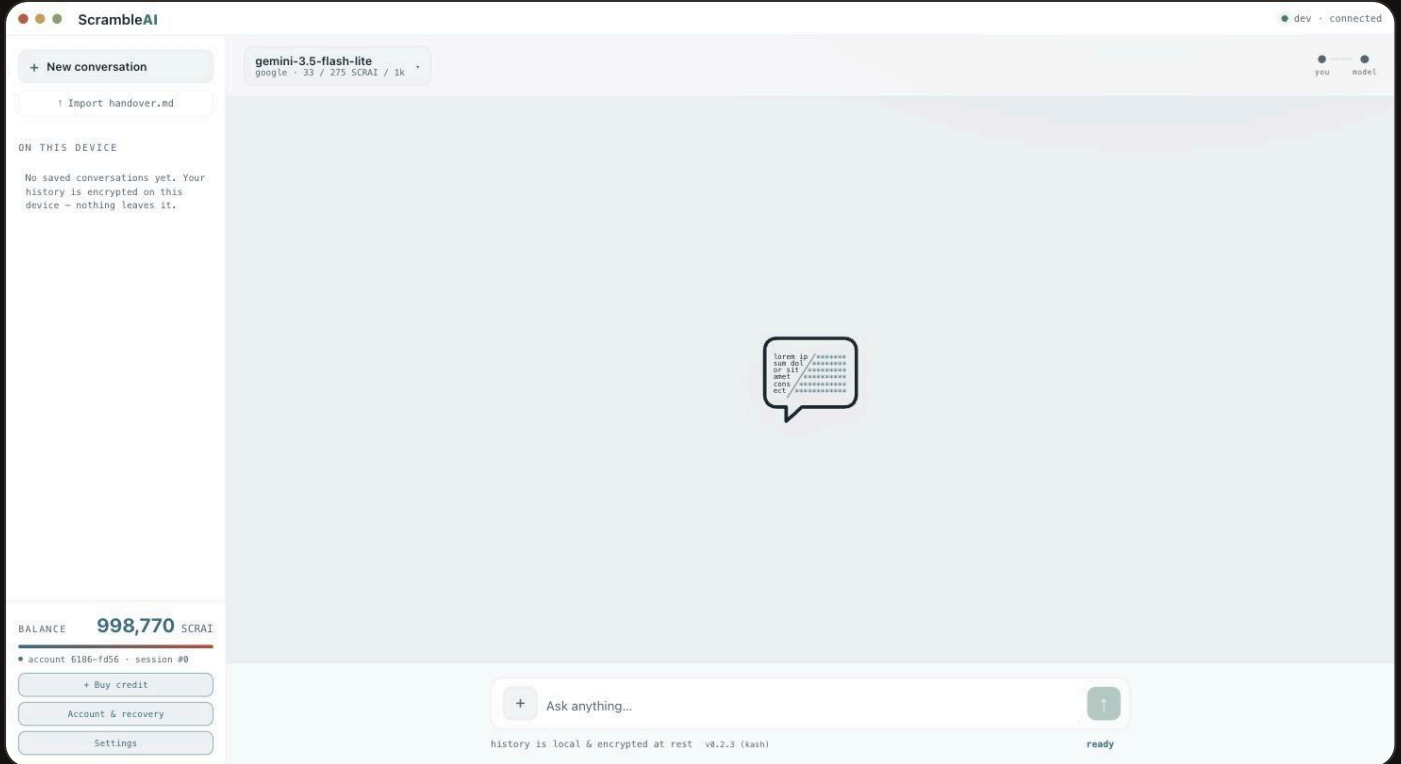
Du wählst, wo du reingehst. Das Entry-Gateway ist nach Land und Host wählbar; die mittleren Mix-Hops bleiben pro Paket neu gewürfelt – genau dieses Mischen *ist* die Anonymität.



Ein Bild kommt an. „Downloading picture · 9/46 chunks“ – jeder Chunk ist eine quittierte Mixnet-Antwort; der erste Chunk markiert das echte First Byte, Generierungs- und Transferzeit werden getrennt sichtbar.



4K, Ende zu Ende. Nano Banana 2 mit 5632 × 3072 px, in 247 s über SURBs geliefert. Die Entwickler-Kostenzeile unter dem Bild: provider \$0.155 · charged \$0.171 · +10 % – die Marge ist eine Zahl, kein Geheimnis.



Vier Themes, eine App. „Nordic frost“, das helle Theme, auf dem Desktop – jede Fläche, auch der Verbinden-Screen, folgt dem gewählten Theme.

Die App passt auf, **bevor** etwas das Gerät verlässt

Anonymes Routing nützt nichts, wenn im Prompt die eigene IBAN steht. Deshalb prüft ScrambleAI jede Nachricht und jeden Anhang **vollständig auf dem Gerät** — nichts davon wird gesendet, geloggt oder gespeichert.

Text-Scan

Mustererkennung für IBANs, Kartennummern, API-Keys, Passwörter, Recovery-Phrasen, Telefonnummern.

Semantischer Scan (lokales Modell)

Ein GLiNER-Modell läuft nativ in der App und erkennt offene Klassen: Namen, Adressen, Arbeitgeber — Zero-Shot, ohne Cloud.

Datei-Scan mit OCR

Bilder und PDFs werden lokal gelesen (Apple Vision nativ, Tesseract-WASM als Fallback) und durch dieselben Prüfungen geschickt. Funde lassen sich direkt im Bild schwärzen.

Manueller Markier-Modus

Die erkannten Boxen sind Startpunkt, nicht Instanz: Fehltreffer per Klick verwerfen, und eigene schwarze Boxen über alles ziehen, was der Scanner nicht kennen kann. Die Balken werden vor dem Senden fest ins Bild eingebrannt.

Metadaten entfernt

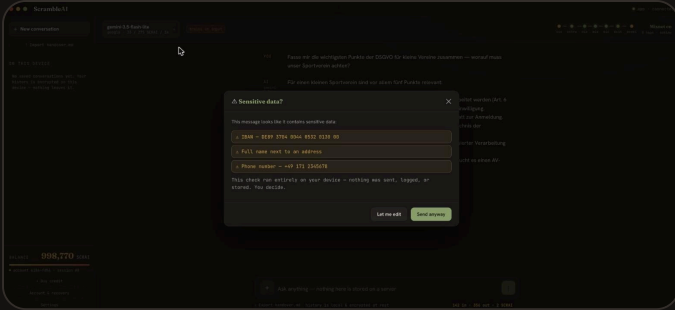
Vor dem Upload wird jedes Bild neu kodiert — EXIF/GPS und das eingebettete Original fallen weg — und auf eine vision-taugliche Größe herunterskaliert, sodass deutlich weniger Bytes durchs Mixnet gehen; PDFs verlieren ihr `/Info`-Dictionary und den XMP-Stream. Nichts verrät dem Modell, wo ein Foto entstand.

Smart Pruning

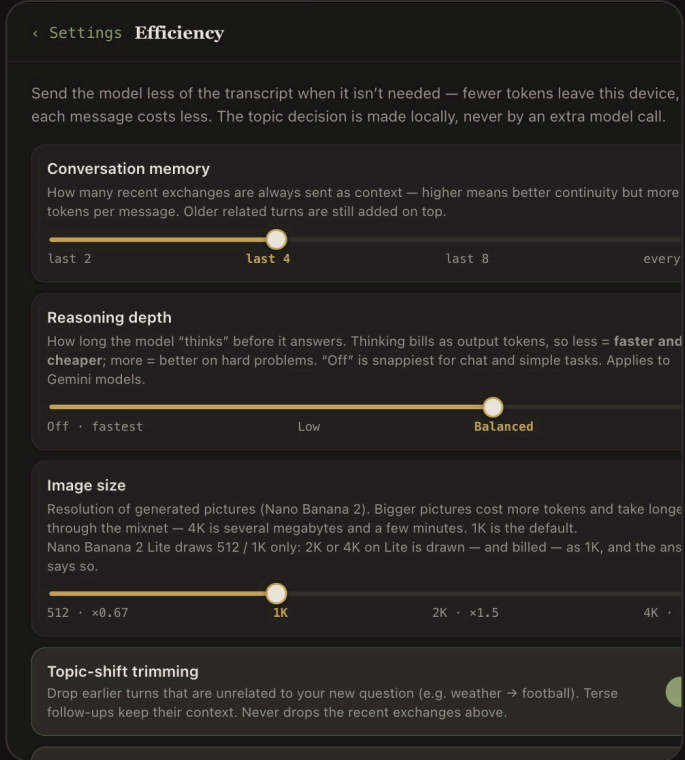
Vor dem Senden wird die Historie lexikalisch auf das Nötige getrimmt — gemessene -60 % Tokens, ohne zusätzlichen Modell-Aufruf. Ein garantierter Gedächtnis-Sockel (die letzten N Wechsel gehen immer mit) hält den Faden, auch wenn die lexikalische Entscheidung ihn kappen würde.

Kosten-Regler

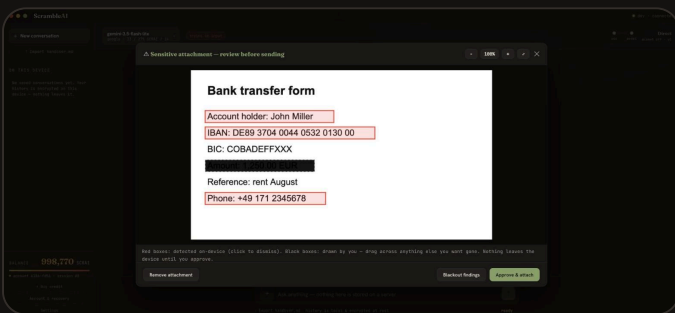
Reasoning-Tiefe (wie lange ein Gemini-Modell nachdenkt — abgerechnet als Output-Tokens) und Bildgröße (512 px → 4K) sind Slider neben den Pruning-Schaltern — Kosten sind eine sichtbare Einstellung, keine Überraschung auf der Rechnung.



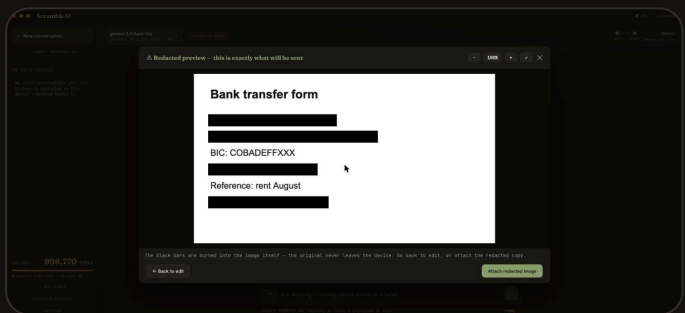
Privacy Guard. Warnt vor IBAN, Name + Adresse, Telefonnummer – „nothing was sent, logged, or stored. You decide.”



Einstellungen > Efficiency. Gedächtnis-Sockel, Reasoning-Tiefe und Bildgröße als Stufen-Slider; die Pruning-Prüfungen darunter einzeln schaltbar. Die Topic-Entscheidung fällt lokal, nie per Modell-Aufruf.



Anhang-Review, Edit-Modus. OCR liest das Formular auf dem Gerät; der Guard markiert Name, IBAN und Telefon (rot). Die schwarze Box über dem Betrag ist von Hand gezogen – der manuelle Modus für alles, was der Scanner nicht als sensibel kennen kann.



Das redigierte Ergebnis. Vorschau vor dem Senden: die Balken sind fest in die Bildkopie eingebrannt – das Original verlässt das Gerät nie.

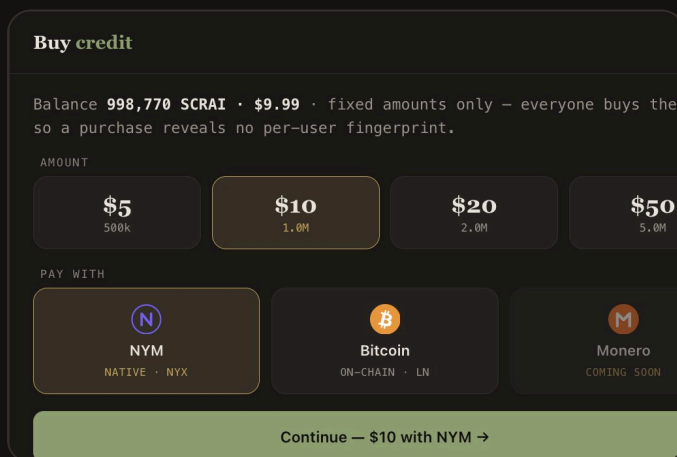
Guthaben, das sich von seinem Käufer trennt

Das Konto ist eine 24-Wort-Phrase (BIP39) — kein Name, keine E-Mail. Gekauft wird in festen Stufen (\$5/10/20/50), damit jeder Kauf aussieht wie jeder andere. Und dann passiert der entscheidende Schritt: das Guthaben wird vom Konto **abgekoppelt**.

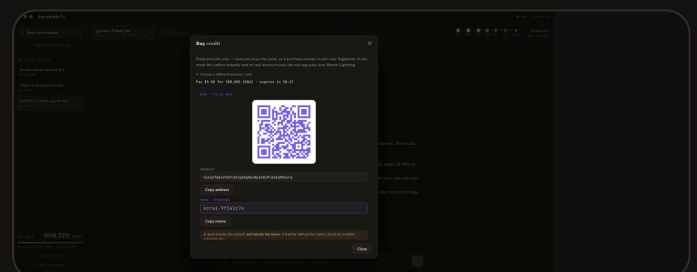
Der Weg des Geldes

- 1 · **Kauf.** Rechnung über Bitcoin/Lightning (BTCPay) oder NYM nativ auf der Nyx-Chain; Monero folgt. Das Konto signiert die Bestellung — hier ist Verknüpfung noch unvermeidbar und ehrlich.
- 2 · **Blind-Abhebung.** Das bezahlte Guthaben wird gegen **Coconut-Credentials** getauscht (zk-nym / nym-compact-ecash, BLS12-381): der Server signiert Ticketbooks, deren Inhalt er *nicht lesen kann*. Die Verbindung Konto → Coins stirbt kryptografisch an dieser Stelle.
- 3 · **Ausgeben.** Die App löst Coins zeitentkoppelt und in uniformen ~\$1-Blöcken in Session-Guthaben ein. Jede Nachricht wird Ed25519-signiert, ihr Preis-Maximum vorab reserviert und nach der Antwort exakt abgerechnet — schlägt der Provider fehl, kostet sie nichts.

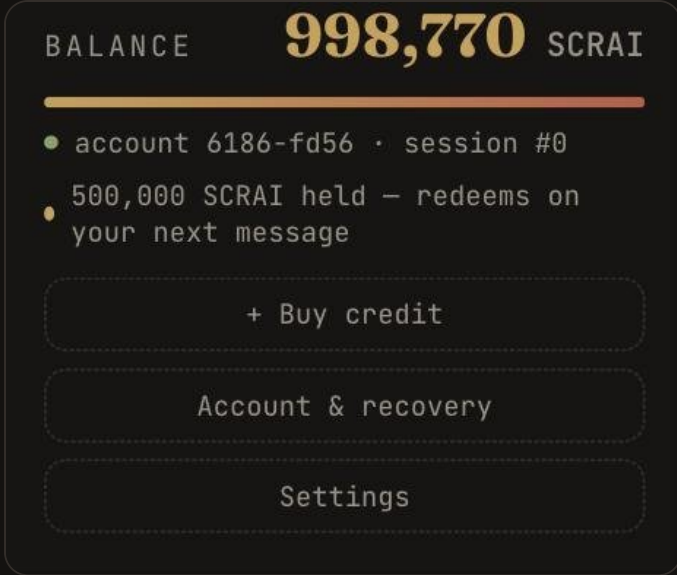
Doppelausgaben werden offline akzeptiert und über ein Quorum erkannt; sanktioniert wird **nur auf kryptografischen Beweis** (zwei widersprüchliche Zahlungen, die den Schlüssel mathematisch enttarnen) — nie auf Verdacht.



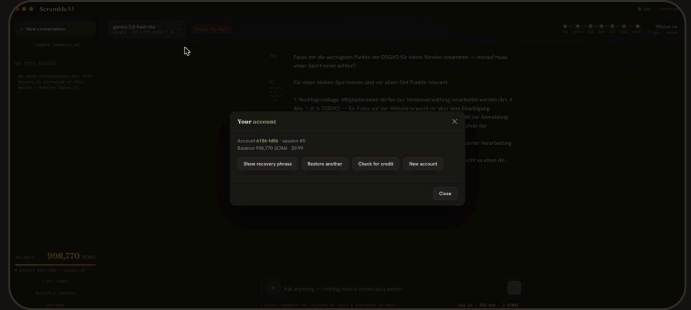
Guthaben kaufen. Feste Beträge, drei Rails: NYM (nativ · Nyx), Bitcoin (On-Chain · LN), Monero (coming soon).



Nativ in NYM bezahlen. \$5 → 500.000 SCRAI auf der Nyx-Chain: lila QR, Adresse, und das Invoice-Memo als eigenes Pflichtfeld — ein Transfer ohne Memo kann nicht zugeordnet werden, also behandelt die UI es genau so. Kurs fixiert, Fenster läuft.



Nach der Zahlung. Das Guthaben liegt als blind signierte Bücher neben der Balance – „500,000 SCRAI held — redeems on your next message“. Kauf und erste Ausgabe bleiben bewusst zeitlich entkoppelt.

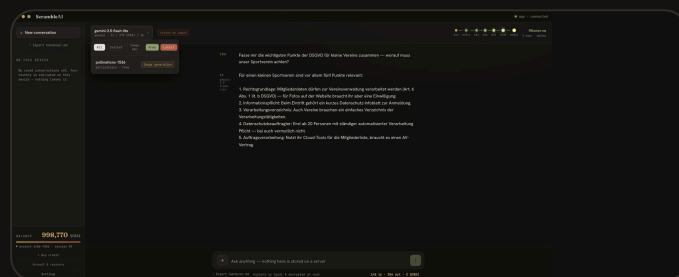
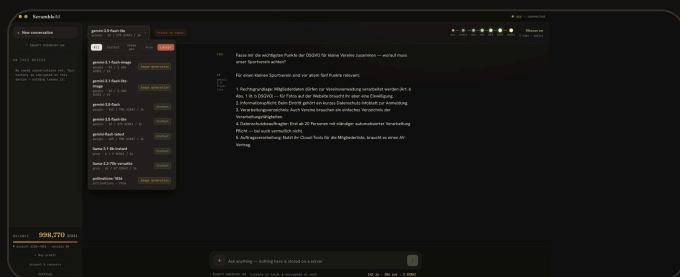


Das Konto. Ein Fingerprint statt eines Namens; Wiederherstellung allein über die Phrase.

Ehrliche Auswahl statt Blackbox

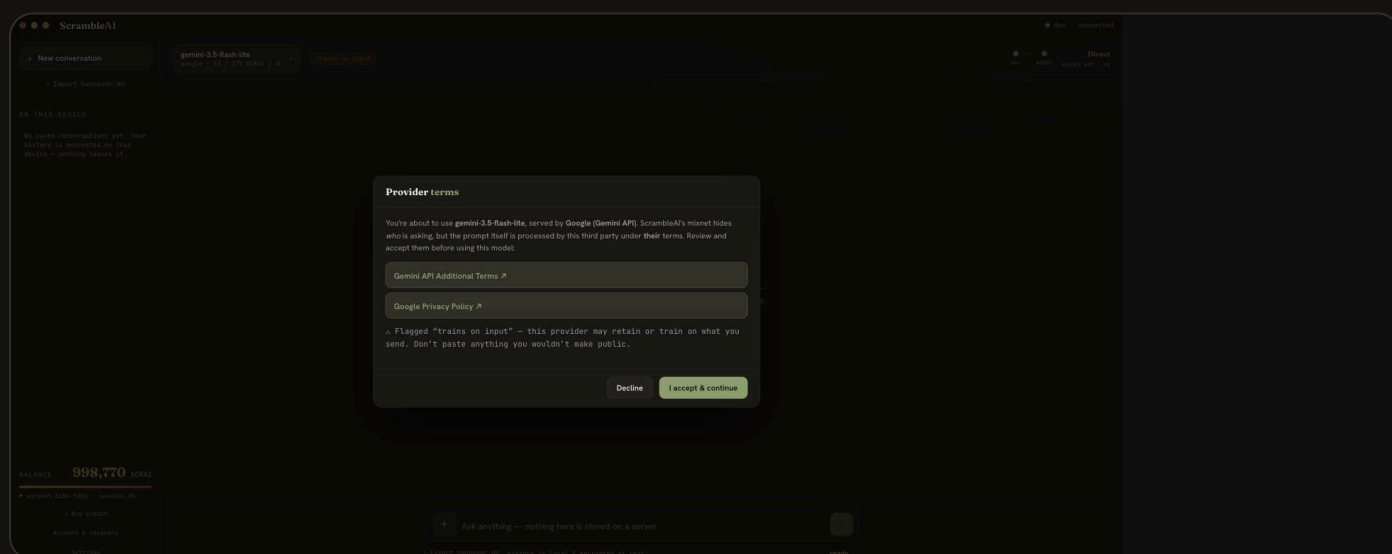
Der Modellkatalog kommt live von den Providern (Google Gemini, Groq — Text und Bild) und wird serverseitig gegen die Preisliste gefiltert: Der Preis im Picker ist der Preis der Abrechnung, ausgewiesen in SCRAI pro 1k Tokens. Modelle, die auf Eingaben trainieren, tragen ein sichtbares **trains on input**-Badge, und vor der ersten Nutzung zeigt die App die Bedingungen des Providers — denn das Mixnet versteckt *wer fragt, nicht bei wem* die Antwort entsteht.

Die Ehrlichkeit reicht bis zur Rechnung und bis zur Absage. Bildmodelle werden so abgerechnet, wie Google uns abrechnet — **Bild-Tokens und Text-/Thinking-Tokens zu ihren getrennten Sätzen**, Zeile für Zeile gegen die veröffentlichte Preisliste geprüft (ein 1K-Bild auf Nano Banana 2 Lite: \$0,034 Provider-Kosten). Wer ein Lite-Modell um 2K bittet, bekommt — und bezahlt — 1K, und die Antwort sagt es. Und lehnt Google eine Anfrage ab (eine erkennbare reale Person, ein Policy-Treffer, ein Thinking-Budget, das die Antwort aufgefressen hat), zeigt die App nicht mehr „no content“: Der Server übersetzt Geminis **finishReason** in einen klaren Satz, der sagt, was zu ändern ist und dass nur das Nachdenken berechnet wurde.



Der Katalog. Filter: All • Chatbot • Image gen • Free • Latest — additiv kombinierbar.

Free-Filter. Nur Modelle, die nichts kosten — und genau so abgerechnet werden.



Provider-Terms. Zustimmung pro Modell, mit Klartext-Warnung, was der Anbieter mit Eingaben tun darf.

YOU

generate an image of charles hoskinson in
blackhawk helicopter

SCRAI

gemini-3.1-flash-lite-image

(the model returned no content)

Eine Absage, ehrlich bepreist. „generate an image of [eine namentlich genannte Person des öffentlichen Lebens] in a helicopter" – Googles Bildmodell zeichnet keine realen Personen. 803 Reasoning-Tokens, null Bild-Tokens, \$0,0012 – und seit diesem Build ein Satz, der den Grund nennt, statt „(the model returned no content)".

Ein Kern, durchgehend Rust

Der komplette Produktionspfad ist Rust — ein geteilter Krypto-Kern, den Desktop-Client und Server byte-kompatibel benutzen.



scrai-core

Geteilte Bibliothek: Coconut-E-Cash, Double-Spend-Quorum, Preistabelle, Billing-Mathematik, Request-Signaturen. Ein Bug-Fix wirkt auf beiden Seiten.

Desktop-App

Tauri 2. Rust-Prozess mit eingebettetem Nym-SDK, nativer OCR und lokalem NER-Modell; die murmur-UI läuft als Web-Frontend im Webview — identisch für die kommenden Mobile-Builds.

scrai-server

Rust-Binary als Nym-Service-Provider.
Persistiert Sessions, Quorum und Paywall in SQLite; Modellkatalog live von Gemini + Groq; Vision-Uploads in quitierten Chunks.

Der Transport, präzise

Offizielles `nym-sdk 1.21.4` auf beiden Seiten, Nym-Mainnet. Die App verbindet sich mit einer **ephemerer Identität** — ein frischer Sender-Key pro Sitzung — und schickt jede Anfrage mit anonymen Reply-SURBs, der Server erfährt nie eine Client-Adresse. Der Server hält eine persistente Nym-Identität, öffnet **keinerlei eingehenden Port** und antwortet nur über die SURBs, die eine Anfrage mitbrachte. Chat, Uploads, Bilder, Zahlungen, sogar der Modellkatalog: ein Pfad, kein Clearnet-Fallback in der App.

Sichtbares Traffic-Shaping

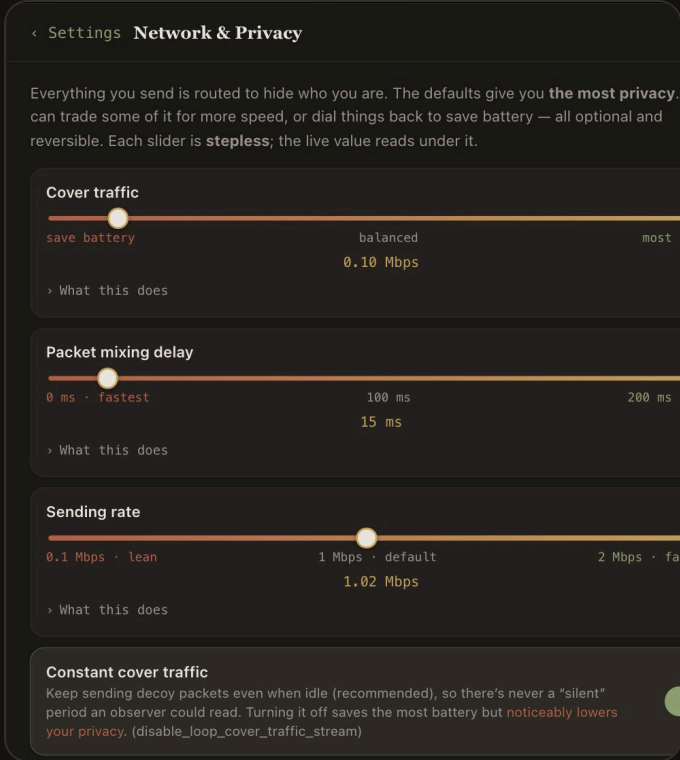
Cover-Traffic, Mixing-Verzögerung pro Hop und Senderate sind Slider unter **Network & Privacy**, voreingestellt auf Nyms eigene Parameter (0,10 Mbps Cover · 15 ms Mixing · 1 Mbps Senden, Cover-Strom immer an). Jeder Regler sagt, was er an Privatsphäre kostet; „Reset to recommended“ ist ein Tipp.

Große Nutzlast, kleine Pakete

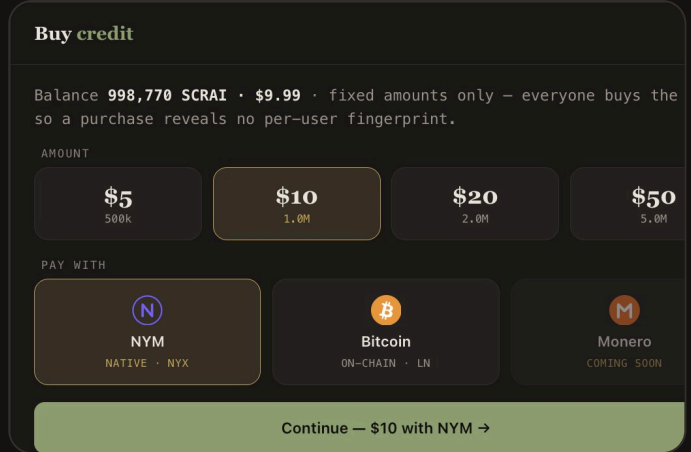
Uploads und generierte Bilder reisen als quittierte 96-KB-Chunks. Downloads laufen mit acht Chunks gleichzeitig — Nym hält nur ~200 Reply-SURBs pro Absender, ein Schwall lässt den Server trockenlaufen — und ein hängender Chunk wird allein neu angefordert. Geparkte Bilder leben 30 min serverseitig, ein abgebrochener Download wird fortgesetzt statt neu berechnet.

Abgesichert & auditiert

Kernpfade unit-getestet (Billing, Signaturen, E-Cash-Lebenszyklus, Upload-Reassemblierung, Absage-Meldungen); Ende-zu-Ende real über das Mixnet verifiziert, auf Desktop und zwei iPhones. Ein Code-Audit am 27. 8. 2026 fand keinen Mock-, Simulations- oder Clearnet-Pfad für Anwendungs-Traffic — und zwei ehrliche Offenlegungen: Die App macht vor dem Verbinden eine Clearnet-Anfrage an Nyms Validator-API für das Gateway-Verzeichnis (derselbe Aufruf, den der Topologie-Fetch des SDK macht), und der heutige Build pinnt eine einzelne Betreiber-Adresse, bis das signierte Verzeichnis kommt.



Einstellungen > Network & Privacy. Die drei Nym-Traffic-Parameter als Slider, Live-Wert jeweils darunter – 0,10 Mbps Cover, 15 ms Mixing, 1,02 Mbps Senden – plus der Schalter für konstanten Cover-Traffic.



Ein Bedienvokabular. Einstellungen, Konto und Guthaben teilen sich eine Karten-, Slider- und Kachelsprache – identisch auf Desktop und Telefon.

Vom einen Server zum Netz, das niemand abschalten kann

Sauber getrennt: was heute schon läuft — und was als Nächstes kommt.

Läuft heute

LIVE

Der komplette anonyme Kreislauf

Chat, Vision-Uploads und Zahlungen Ende-zu-Ende über das echte Mixnet: kontosignierte Käufe (natives NYM auf Nyx mit Memo-Zuordnung über den eigenen Chain-Watcher, dazu Bitcoin/Lightning via BTCPay), blind signierte Coconut-Ticketbooks, getaktete Sessions mit Reserve → Settle pro Nachricht, und ein bezahlschrankengesicherter Modellkatalog live von den Providern.

LIVE

Megabyte-Antworten

Bildgenerierung bis 4K über das Mixnet — gehunkt, gefenstert, fortsetzbar — mit echtem Fortschrittsbalken und Google-exakter Abrechnung (Bild-Tokens vs. Text-Tokens). Auf dem iPhone verifiziert: 5632 × 3072 px in 247 s. Text- und Bildmodelle von Google, Groq und Cloudflare, dazu ein schlüsselloser Gratis-Pfad zum Testen.

LIVE

Föderationsreife Kryptografie

Die Coconut-Schicht ist von Anfang an t-von-n — Threshold-Issuance mit einem aggregierten Verification Key ist im geteilten Kern implementiert und getestet; das aktuelle Deployment fährt sie schlicht als 1-von-1 — und das sagen wir klar: Die einzelne Authority ist bis zur Threshold-Key-Zeremonie für die Ausgabe vertrauenswürdig zu setzen, und der Server startet in diesem Modus nur mit ausdrücklicher Betreiber-Freigabe. Die Skalierung auf viele scrai-server, bei der ein Guthaben überall gilt, ist Deployment und Key-Zeremonie, kein Rewrite. Double-Spenden werden offline akzeptiert und vom Mehrheits-Quorum erkannt; sanktioniert wird nur auf identify()-Beweis.

NEXT

Server-Discovery ohne Erlaubnis

Jeder Server announced sich mit einem **signierten Descriptor** — Nym-Adresse, Föderations-Key, Modellliste, signierte Preisliste — in einer Registry auf der **Nyx-Chain** (ein Announce kostet eine kleine Gebühr: eingebauter Spam-Schutz), repliziert per Gossip zwischen den Föderationsmitgliedern. Die App lädt das Verzeichnis übers Mixnet und verifiziert jeden Eintrag gegen den Föderations-Key: **Preise sind clientseitig beweisbar** — ein Server kann weder Sonderpreise pro Kunde noch stille Erhöhungen unterschieben.

NEXT

Mehr Rails, mehr Durchsatz

Monero als dritter Zahlungsweg und Ticketbooks in Kaufstufen-Größe. Serverseitig ist der Engpass Nyms Pacing pro Client (≈ 100 KB/s Egress bei Standardwerten — für Text reichlich, bei gleichzeitigen 4K-Bildern eine Warteschlange): Als Nächstes kommen eine getunte Server-Senderate und mehrere Nym-Identitäten pro Server, angekündigt über dasselbe signierte Verzeichnis.

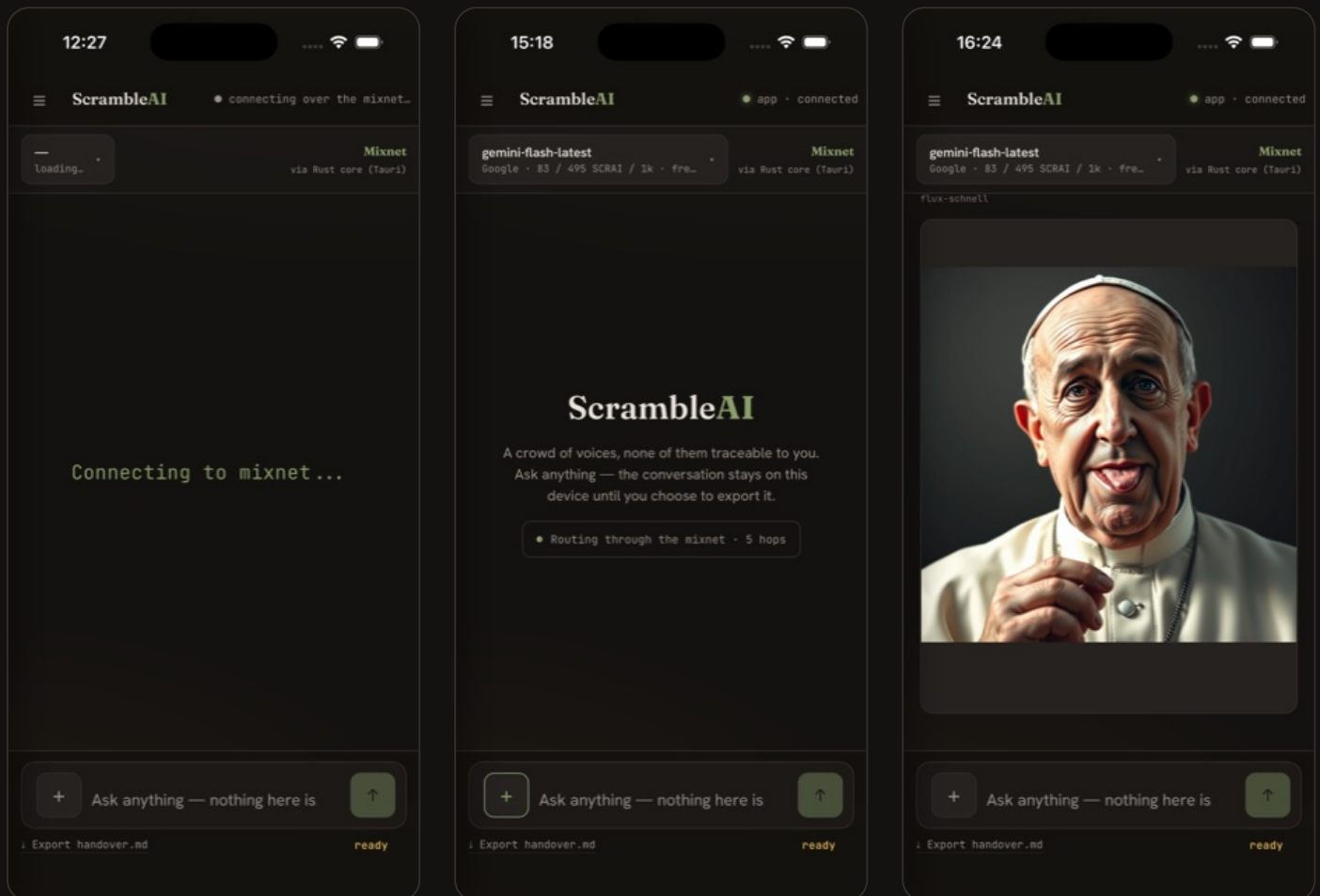
NEXT

Gleicher Kern, jede Plattform

iOS läuft bereits mit dem vollen Stack — siehe das Kapitel unten. Android folgt über dasselbe Tauri-2-Setup mit demselben Rust-Kern und derselben UI; Schlüssel wandern in die OS-Keychain.

Dieselbe App, in der Hosentasche

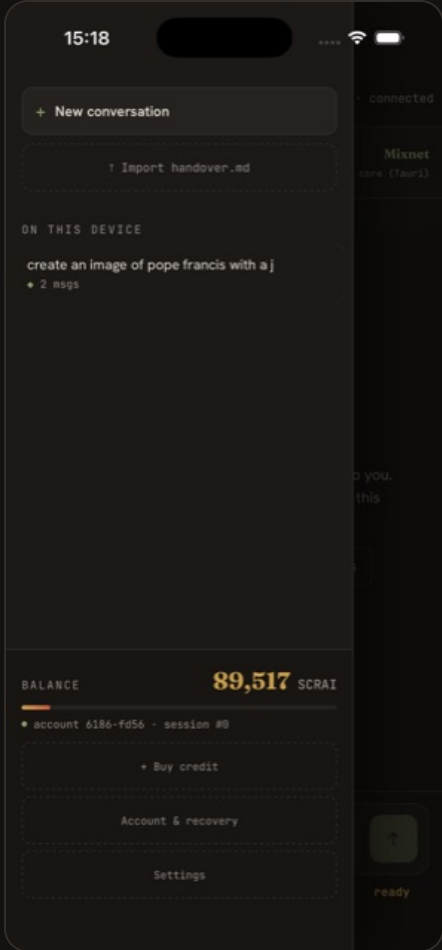
Keine zweite Codebase: der identische Rust-Kern — Nym SDK, Coconut-Wallet, Privacy Guard, native OCR — für iOS kompiliert mit Tauri 2, mit derselben UI. Chat, Zahlungen und Bildgenerierung — bis 4K — laufen heute vom Telefon aus übers Mixnet, auf echten Geräten (iPhone 15 und 17e im täglichen Einsatz). Ein Tutorial beim ersten Start führt durch Account → Server → Guthaben → Chat, der Verbinden-Screen benennt jeden echten Schritt der Route, während er live steht, und vier Themes stylen die ganze App um — inklusive Verbinden-Screen. Generierte Bilder landen in Fotos, Exporte gehen über das Share-Sheet, und die Recovery-Phrase wird nur hinter Face ID gezeigt, nativ — sie betritt den Webview nie.



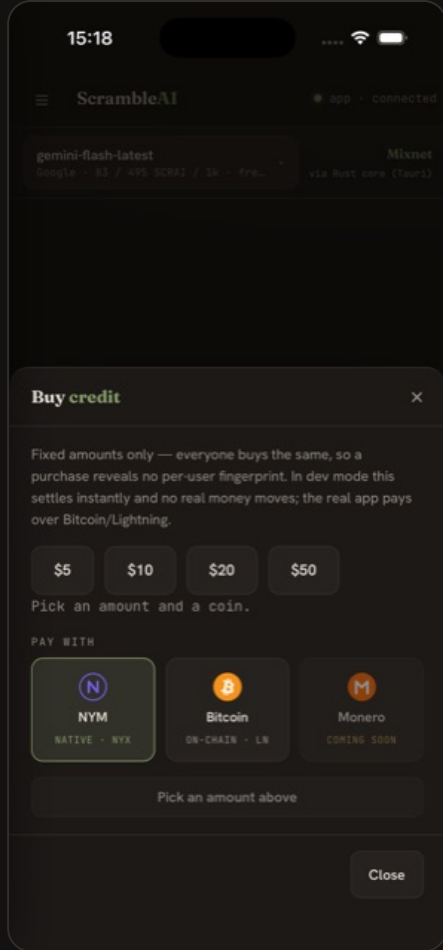
Verbinden. Der eingebettete Nym-Client baut beim Start seine Route.

Bereit. Routing durchs Mixnet • 5 Hops.

Bildgenerierung. flux-schnell, eine Megabyte-Antwort über SURBs.



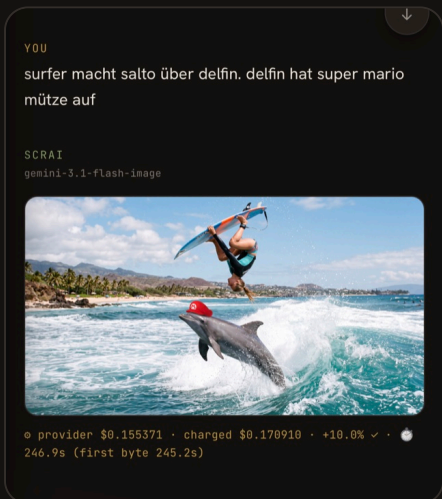
On this device. Verschlüsselte lokale Historie, darunter das SCRAI-Guthaben.



Guthaben kaufen. Feste Stufen als Bottom Sheet – Nym nativ, Bitcoin, Monero als Nächstes.



Einstellungen. Smart Pruning und der On-Device-Privacy-Guard – dieselben Schalter, in Telefongröße.



4K vom Telefon. Dieselbe gehunkte, fortsetzbare Zustellung – 46 Chunks über SURBs, ein Fortschrittsbalken, während sie landen.

ScrambleAI · anonyme KI über das Nym-Mixnet

Zahlen: Mixnet-RTT und Pruning-Ersparnis real gemessen · Zahlungen aktuell Testnet